Preprocessing Experimental Spike Recordings for PRISM-EM

Ver3c biological-data input preparation

Jan Balewski, NERSC

June 16, 2026

Abstract

This document describes the current `prep_bioexp3c.py` preprocessing program for biological spike recordings. The script reads raw experimental spike times and a per-channel metrics spreadsheet, converts spike times into a binned spike-count matrix, filters channels by firing-rate range, orders accepted neurons by firing rate, and writes two companion files. The first file, `.spikes.npz`, contains only the data needed by the EM fitter. The second file, `.bioExp.npz`, contains experimental metadata, MEA identifiers, metrics-table rows, and other biological bookkeeping used by inspection, plotting, and downstream interpretation.

Contents

1 Purpose and Scope

prep_bioexp3c.py converts one experimental recording session into the same spike-file contract used elsewhere in the ver3c pipeline: rows are time bins and columns are neurons. The preprocessing script does not fit a model and does not estimate connectivity. Its job is to make the experimental recording usable by the EM fitter while keeping experiment-specific information outside the fitter input.

The key design choice is a two-file output:

- <dataName>.spikes.npz: compact fitter input;
- <dataName>.bioExp.npz: biological/session metadata and channel annotations.

The files share the same stem and are written to the same directory.

2 Command-Line Inputs

The command-line arguments are:

- --expPath: root directory containing the raw experimental session directory;
- --sessionName: session path below --expPath;
- --dataPath: output directory for both generated .npz files;
- --shortName: optional output stem. If omitted, the script uses <recording_date>-<6hex>;
- --inpFormat: metrics-spreadsheet selector, 1 for metrics_curated.xlsx and 2 for quality_metrics.xlsx;
- --samp_freq: binning frequency in Hz, default 100;
- --freqRange: inclusive accepted firing-rate range in Hz, default 1 50;
- --verb: diagnostic verbosity.

The raw session is expected to contain

<expPath>/<sessionName>/spike_times.npy

and one metrics spreadsheet:

metrics_curated.xlsx or quality_metrics.xlsx.

The session name is parsed as a slash-separated path whose fields include cell line, recording date, chip ID, run number, and well number. These values are stored in the biological metadata.

3 Preprocessing Steps

3.1 Read Raw Spike Times

The input spike_times.npy is loaded as a Python dictionary keyed by MEA feature/channel identifiers. The keys are sorted once, and this sorted key order defines the initial raw channel order.

For each channel, spike times are multiplied by --samp_freq and cast to integer bin indices. Thus --samp_freq=100 corresponds to $\Delta t = 0.01\,\mathrm{s}$. Repeated bin indices are retained and later become spike counts larger than one in a bin.

The script computes:

- the total number of time bins;
- the recording duration in seconds;
- one firing rate per raw channel.

3.2 Filter by Firing Rate

Channels are retained if their firing rate lies inside the inclusive range

$$\texttt{freqRange[0]} \leq r_i \leq \texttt{freqRange[1]}.$$

The metadata records how many channels were dropped below and above this range. The remaining channels define the population used downstream.

3.3 Sort Accepted Neurons by Firing Rate

After rate filtering, accepted channels are sorted by firing rate in ascending order. This frequency-sorted order becomes the primary neuron index used in the saved spike matrix. The `.bioExp.npz` file stores both the frequency-sort index and the inverse map so that downstream tools can relate the saved columns back to the accepted-channel order.

3.4 Build the Spike-Count Matrix

For each accepted, frequency-sorted channel, the script forms a length-T spike-count vector using `numpy.bincount`. These vectors are stacked into

$$Y \in \mathbb{N}_0^{T \times N},$$

where rows are time bins and columns are accepted neurons.

The maximum pre-clipping count in any bin is recorded in the biological metadata. The matrix written to the fitter-facing file is clipped to $[0, 255]$ and stored as `uint8`. This keeps the spike file compact while preserving ordinary binned spike counts.

3.5 Load and Align Metrics

The metrics spreadsheet is loaded with `pandas`. For input format 1, the file name is `metrics_curated.xlsx`. For input format 2, the file name is `quality_metrics.xlsx`; location columns named `location_X`, `location_Y`, and `location_Z` are renamed to `loc_x`, `loc_y`, and `loc_z` when present.

Rows are matched by `MEA_idx` and reordered to match the final frequency-sorted neuron order. Duplicate `MEA_idx` rows are treated as an error, and every retained channel must have a corresponding metrics row.

3.6 Compute Summary Statistics

The script computes summary statistics from the accepted channels: mean, standard deviation, median, minimum, and maximum firing rate, plus the mean and standard deviation of the per-neuron Fano factor. These values are stored in the biological metadata.

4 Output File Pattern

Let `<dataName>` denote the output stem. It is equal to `--shortName` when that argument is supplied; otherwise it is generated as

`<recording_date>-<6hex>`.

The script writes both output files directly into `--dataPath`:

`<dataPath>/<dataName>.bioExp.npz`, `<dataPath>/<dataName>.spikes.npz`.

The order is deliberate: the `.spikes.npz` file is the direct input to numerical fitters, while the `.bioExp.npz` file stores experimental context that should not be required by the EM training code.

5 Saved Contents

5.1 <dataName>.spikes.npz

The fitter-facing data dictionary contains:

- spikes: uint8 array with shape (T, N);
- single_rates: floating-point array with shape $(N,)$, ordered exactly like the spike-matrix columns.

The metadata contains:

- time_step_sec: bin width, equal to 1/samp_freq;
- provenance.experiment_name: the output stem <dataName>;
- poisson_eta_clip: clipping value expected by the EM fitter, currently 5;
- data_type: bioExp;
- num_neurons: number of accepted channels N.

5.2 <dataName>.bioExp.npz

The biological data dictionary contains:

- neur_freqIdx: index vector that sorts accepted channels by firing rate;
- neur_revFreqIdx: inverse map from accepted-channel order to frequency-sorted order;
- single_rates: firing rates in the final saved neuron order;
- MEA_idx: MEA identifiers reordered to match the final neuron order;
- spike_key: raw spike_times.npy keys reordered to match the final neuron order;
- metrics_curated: metrics-spreadsheet rows reordered to match the final neuron order.

The biological metadata contains:

- bioexp: raw-data provenance such as raw_bioexp_path, session_name, inp_format, cell_line_name, recording_date, chip_ID, run_num, well_num, sampling_freq, num_feature, num_time_bin, and max_time;
- data_selector: selection parameters and results, including freq_range, num_drop_neur_lo_hi_freq, num_chan, and max_spike_per_bin;
- hash: random six-character hash used when --shortName is omitted;
- short_name: output stem <dataName>;
- metrics_curated_columns: spreadsheet column names after any input-format normalization;
- rate_summary: firing-rate and Fano-factor summary statistics for the accepted population.

6 Example Commands

For inspection or plotting only, the output directory can be any directory chosen by `--dataPath`. For use with the ver3c EM fitter, choose the same `spikesData` layout used by `fitEM_states_ver3c.tex`:

```
basePath=/path/to/analysis

./prep_bioexp3c.py \
  --expPath /path/to/raw/experiments \
  --sessionName <cellLine>/<recordingDate>/<chipID>/<assay>/<run>/<well> \
  --dataPath $basePath/spikesData \
  --shortName <dataName> \
  --inpFormat 1 \
  --samp_freq 100 \
  --freqRange 1 50
```

This produces:

`$basePath/spikesData/<dataName>.spikes.npz`, `$basePath/spikesData/<dataName>.bioExp.npz`.

The `.spikes.npz` file is then the input selected by `--basePath $basePath --dataName <dataName>` in the EM training workflow. See `fitEM_states_ver3c.tex` for the EM model, fitting options, and fit-output file pattern.